PHENIKAA UNIVERSITY

PHENIKAA SCHOOL OF INFORMATION TECHNOLOGY

COURSE: SOFTWARE ARCHITECTURE

PROJECT TITLE: Development of an Online Movie Streaming Platform (Nozie Platform)

Instructor:	Vũ Quang Dũng
Team Members:	Nguyễn Văn Nhật - 23010887
	Nghiêm Đức Việt - 23010636
	Nguyễn Duy Minh - 23010359
	Đặng Nhất Nhất - 23010345
Class Code:	CSE703110-1-2-25(N02)

Hà Nội, tháng 3 năm 2026

Mục Lục

1. Cover Page & General Information

1.1 Course and Instructor Information

This project was developed within the framework of the **Software Architecture** course for the **Second Semester/Academic Year 2025-2026** at **Phenikaa University/ Faculty of Information Technology**.

Under the guidance of **Vũ Quang Dũng,** the course aims to facilitate the practical application of core architectural concepts—including layered architecture, microservices, event-driven architecture (EDA), and deployment views—to a realistic software system.

1.2 Team & Member Information

The project is implemented by the **Nozie Team**, comprising the following members:

- **Nguyễn Văn Nhật – 23010887:** Team Lead. Responsible for overall system architecture and backend integration.
- **Nghiêm Đức Việt – 23010636**: Backend Developer. In charge of microservices implementation and database design.
- **Nguyễn Duy Minh – 23010359:** Frontend/Flutter Developer. Responsible for the mobile client application and user experience (UX).
- **Đặng Nhất Nhất – 23010345:** QA & Documentation. Responsible for reports, architectural diagrams, and test evidence.

All members contributed to both design and implementation, with specific ownership of labs and report sections defined in the team's planning document.

1.3 System Name and Description

System Name: Nozie Movie Platform

Description: Nozie is an online movie streaming platform designed to allow users to browse a vast catalog of movies, subscribe to premium plans, and stream content based on their subscription tier. The system is engineered as a suite of microservices (**Identity, Customer, Movie, Payment, Notification**) exposed via a centralized **API Gateway** and consumed by a cross-platform **Flutter** client application.

2. Executive Summary

2.1 Overall Project Objectives

The primary objective of the Nozie project is to design and implement a **production-grade movie streaming platform** that demonstrates mastery of key software architecture concepts.

The system is designed to support core workflows—including user registration, catalog browsing, subscription management, and notifications—while satisfying critical architectural drivers such as **scalability, availability, and modifiability**.

2.2 Main Functional Scope

User Features:

- **Authentication:** Secure registration and login functionality.
- **Catalog Management:** Browse, search, and view detailed movie information.
- **Subscription Management:** Purchase and manage plans (e.g., Free, Premium).
- **Streaming:** Access and watch content restricted by current subscription levels.
- **Notifications:** Receive alerts regarding registration, subscription activation, and payments.

Administrative Features:

- Maintain consistent customer profiles and verify subscription statuses across distributed services.

2.3 Key Architectural Decisions

The project adopts a **Microservice Architecture** pattern, ensuring loose coupling and high cohesion. Key technical decisions include:

- **Database-per-Service:** Each microservice maintains its own data sovereignty.
- **API Gateway:** Acts as the single entry point for the Flutter client, routing requests to appropriate backend services.
- **Hybrid Communication:**
    - **Synchronous:** REST APIs are used for direct client-to-gateway and gateway-to-service communication.
    - **Asynchronous (Event-Driven): RabbitMQ** is utilized to decouple services (e.g., Payment, Customer, and Notification), ensuring resilience and scalability.
- **Layered Internal Architecture:** Each service follows a strict separation of concerns (Controller → Service → Repository).
- **Scalable Deployment:** The deployment view is modeled to support independent scaling of high-load services, such as Movie and Payment.

2.4 Main Outcomes and Final Demo

The project concludes with a fully functional, **end-to-end system**. The final demonstration validates that a user can successfully:

1. Register and authenticate.
2. Browse the movie catalog.
3. Select a subscription plan and complete the payment flow.
4. Experience immediate unlocking of premium content via real-time status updates.

3. Project Requirements & Goals

3.1 Problem Background & Business Context

The **Nozie Movie Platform** addresses the market demand for a flexible and scalable online streaming service that allows users to discover content, subscribe to tiered plans, and stream across multiple devices.

Traditional monolithic architectures often hinder rapid evolution, making it difficult to introduce new capabilities—such as advanced recommendation engines, promotional campaigns, or diverse payment methods—without risking system stability. Consequently, this project focuses not merely on delivering basic streaming functionality, but on engineering an architecture capable of evolving alongside future business demands (e.g., new subscription tiers, partner integrations, and regional catalogs) while maintaining loose coupling between teams and services.

3.2 Functional Requirements (FRs)

The system is designed to support the following core functional flows:

- **FR-01: Registration & Authentication** Users must be able to securely register and log in to the system.
- **FR-02: Catalog Management** Users must be able to browse, search the movie catalog, and view detailed metadata for specific titles.
- **FR-03: Subscription & Payment** The system must allow users to select subscription plans (e.g., Free, Premium, VIP) and initiate secure payment flows.
- **FR-04: Access Control & Enforcement** Upon successful payment, the user's subscription status must be updated immediately and strictly enforced when attempting to access content.
- **FR-05: System Notifications** The system must trigger automated notifications regarding registration, subscription activation, and payment receipts.

3.3 Non-Functional Requirements (NFRs)

The architecture is driven by the following critical quality attributes:

- **Scalability:** The system must handle traffic spikes—such as concurrent browsing or high-volume subscription requests—by allowing specific services (e.g., Movie, Payment) to scale horizontally.
- **Availability & Resilience:** The architecture must ensure fault isolation. Failures in non-critical services (e.g., Notification) must not block core business flows like authentication or payment processing; the system must degrade gracefully.
- **Performance:** Critical user interactions, particularly catalog browsing and checkout completion, must exhibit low latency and high responsiveness.
- **Security:** All APIs accessible via the API Gateway must be protected by robust Authentication and Authorization mechanisms. Sensitive data, particularly payment information, must be handled according to security best practices.
- **Modifiability:** The architecture must facilitate the addition of new features—such as new subscription tiers, notification channels, or auxiliary services—with minimal impact on existing codebases.

3.4 Architecturally Significant Requirements (ASRs)

Based on the defined FRs and NFRs, several requirements have directly influenced the architectural topology:

- **ASR-1: High Scalability for Read-Heavy Traffic**
  - *Impact:* The Movie catalog requires horizontal scaling to handle read-heavy loads. This necessitates a dedicated **Movie Microservice** with its own database and independent deployment units.
- **ASR-2: Decoupled Payment Side-Effects**
  - *Impact:* Payment completion must not be blocked by downstream processes (e.g., sending emails or updating read models). This drives the adoption of an **Event-Driven Architecture** using **RabbitMQ** for asynchronous consistency.
- **ASR-3: Independent Evolution of Domains**
  - *Impact:* Identity, Customer, Payment, Movie, and Notification domains must evolve independently with separate release cycles. This reinforces the **Database-per-Service** pattern and the definition of clear service contracts.
- **ASR-4: Secure, Single Entry Point**
  - *Impact:* All external client traffic must be routed through a centralized **API Gateway**. This centralizes cross-cutting concerns such as authentication, routing, and logging, simplifying the client-side logic.

3.5 Use Case Model & Actors

The system interaction involves the following primary actors:

- **End User (Customer):** The primary user who registers, authenticates, browses the catalog, manages subscription plans, executes payments, and consumes streaming content based on their access level.
- **Content/Platform Administrator:** The internal user responsible for managing the movie catalog metadata and monitoring subscription health (note: *while modeled, full implementation of the admin dashboard is secondary to the consumer-facing features*).

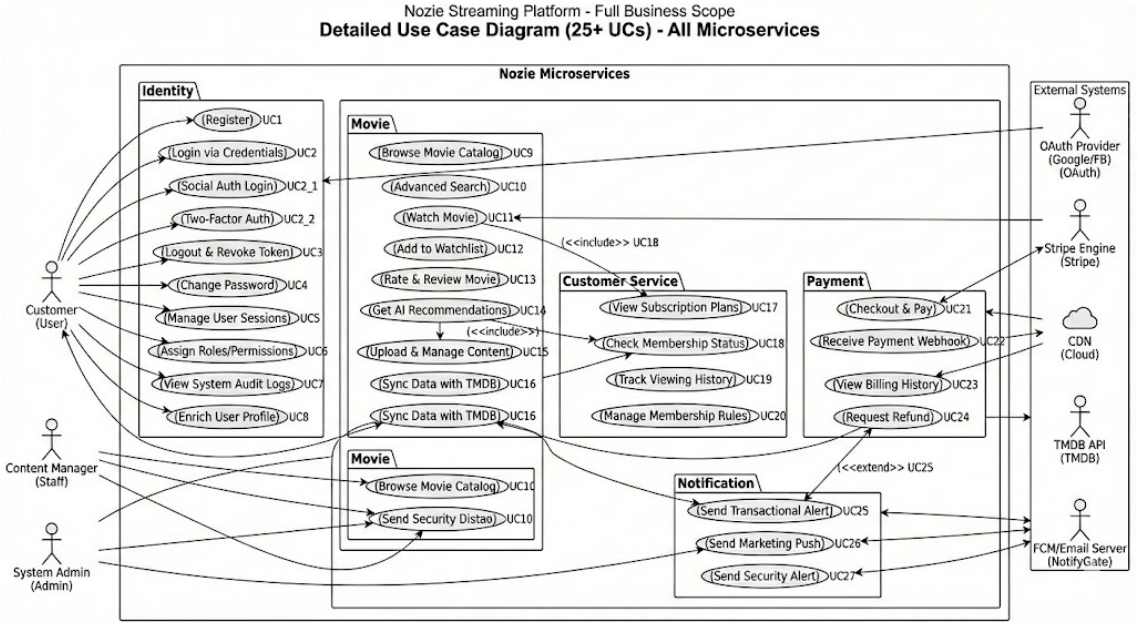


Figure 3.5 Usecase Diagram

Use Case Modeling:

A. Identity Domain (UC1–UC8)

Handles authentication, authorization, and user account management.

Use Case	Actor	Flow Description
UC1 – Register	Customer	1. User submits registration details. 2. Identity Service creates account in DB. 3. Publishes UserRegisteredEvent. 4. Returns success to Client.
UC2 – Login via Credentials	Customer	1. User inputs email/password. 2. Identity Service validates hash. 3. If valid, issues JWT Access & Refresh Tokens. 4. Client stores tokens securely.
UC2_1 – Social Auth Login	Customer	1. User selects "Login with Google/Facebook". 2. Client handles OAuth flow with Provider. 3. Identity Service verifies Provider Token. 4. Service creates/links account and issues App JWT.
UC2_2 – Two-Factor Auth	Customer	1. After initial credential check, System requests 2FA code. 2. User enters code (SMS/Authenticator). 3. Identity Service validates code. 4. Issues final Access Token.
UC3 – Logout & Revoke	Customer	1. User selects Logout. 2. Client discards local token. 3. (Optional) Client calls API to blacklist/revoke the specific Refresh Token on server.
UC4 – Change Password	Customer	1. User provides old and new passwords. 2. Identity Service validates old password. 3. Updates DB with new hash. 4. Triggers security notification (UC27).
UC5 – Manage Sessions	Customer	1. User requests active session list. 2. Identity Service returns list of devices/IPs. 3. User can revoke specific unknown sessions.
UC6 – Assign Roles	Admin	1. Admin selects a user and a role (e.g., Staff, Manager). 2. Identity Service updates user claims. 3. Permissions take effect on next token refresh.
UC7 – View Audit Logs	Admin	1. Admin requests system logs. 2. Identity Service returns history of logins, failed attempts, and role changes for security auditing.

Use Case	Actor	Flow Description
UC8 – Enrich Profile	Customer	1. User views/edits profile (Avatar, Name, Bio). 2. Client calls PUT /api/users/me. 3. Identity Service updates details.

B. Movie Domain (UC9–UC16)
Handles catalog management, streaming, and content discovery.

Use Case	Actor	Flow Description
UC9 – Browse Catalog	Customer	1. Client requests movie list (Latest, Trending). 2. Movie Service queries DB with pagination. 3. Returns metadata list (Posters, Titles).
UC10 – Advanced Search	Customer	1. User searches by multiple criteria (Actor, Year, Genre). 2. Movie Service executes complex query. 3. Returns filtered results.
UC11 – Watch Movie	Customer	1. User clicks Play. 2. System calls **UC18** to verify Subscription. 3. If allowed, Movie Service returns Stream URL/Manifest. 4. Player begins streaming.
UC12 – Add to Watchlist	Customer	1. User adds a title to "My List". 2. Movie Service stores the userId + movieId association. 3. Client updates UI to show "Added".
UC13 – Rate & Review	Customer	1. User rates (1-5 stars) and writes a review. 2. Movie Service saves review. 3. Updates average rating for the movie.
UC14 – Get AI Recs	Customer	*(Future/Stub)* 1. System analyzes user history. 2. Calls AI Model to get recommended IDs. 3. Movie Service returns personalized list.
UC15 – Upload Content	Staff	1. Staff uploads video file and metadata. 2. Movie Service processes file (transcoding) and saves details to DB. 3. Movie becomes available in catalog.
UC16 – Sync TMDB	Staff	*(Future/Stub)* 1. Staff triggers sync. 2. Service fetches metadata from TMDB API. 3. Updates local DB to match external source.

C. Customer Domain (UC17–UC20)
Handles subscription logic, history, and customer-specific business rules.

Use Case	Actor	Flow Description
UC17 – View Plans	Customer	1. Client requests available subscription tiers. 2. Service returns Plan list (Free, Premium, VIP) with features and pricing.
UC18 – Check Status	System	1. Internal/External request to check if User X can access Plan Y. 2. Service checks subscriptionStatus and expiryDate. 3. Returns TRUE/FALSE.
UC19 – Track History	Customer	1. As user watches (UC11), system logs progress. 2. User requests "History". 3. Service returns list of recently watched items and timestamps.
UC20 – Manage Rules	Admin	1. Admin configures mapping between Plans and Permissions (e.g., "VIP" = 4K support). 2. Service updates business rule configuration.

D. Payment Domain (UC21–UC24)

Handles billing, integration with payment gateways (Stripe), and transactions.

Use Case	Actor	Flow Description
UC21 – Checkout & Pay	Customer	1. User selects plan. 2. Payment Service creates intent with Provider (Stripe). 3. User completes payment UI. 4. Transaction recorded as "Pending".
UC22 – Receive Webhook	System	1. Payment Provider calls Webhook URL. 2. Payment Service validates signature. 3. Updates transaction to "Success". 4. Publishes SubscriptionActivatedEvent (EDA).
UC23 – Billing History	Customer	1. User requests billing logs. 2. Payment Service returns list of past invoices/payments (Date, Amount, Plan).
UC24 – Request Refund	Customer	1. User requests refund for eligible transaction. 2. System validates policy (e.g., < 7 days). 3. Payment Service triggers refund via Provider API.

E. Notification Domain (UC25–UC27)

Handles asynchronous communication via Email/Push.

Use Case	Actor	Flow Description
UC25 – Transactional Alert	System	1. Service consumes RabbitMQ events (e.g., PaymentSucceeded). 2. Generates email template. 3. Sends confirmation email to user.

Use Case	Actor	Flow Description
UC26 – Marketing Push	Admin	1. Admin drafts a promo message. 2. Notification Service queues message for all/selected users. 3. Sends Push Notification/Email batch.
UC27 – Security Alert	System	1. Triggered by **UC2_2** or **UC4**. 2. Service sends "New Login Detected" or "Password Changed" email to protect user account.

4. Architectural Design

4.1 Layered Architecture
DDD-Oriented Layered Architecture:

Nozie's backend services follow a **DDD-oriented layered architecture**, ensuring clear separation between API concerns, use cases, domain logic, and infrastructure. Each microservice is organized into four distinct layers:

- **API Layer (Interface / Controller)** This layer exposes the service via REST endpoints. Controllers (e.g., `PaymentController`) handle HTTP requests, input validation, and DTO mapping. This layer contains **no business rules**, delegating all processing to the Application Layer.

- **Application Layer (Use-Case)** Implements specific service use cases such as "Checkout" or "Activate Subscription." Application Services coordinate domain objects and repositories, manage transaction boundaries, and publish integration events. This layer depends on the Domain Layer but remains **independent** of infrastructure details like JPA or RabbitMQ.

- **Domain Layer (Domain Model)** Captures core business logic using rich domain models. It defines **Aggregates** (e.g., `Subscription`, `Payment`), **Value Objects** (e.g., `Money`), and declares **Repository Interfaces**. This layer consists of pure domain code, entirely isolated from frameworks and technical concerns.

- **Infrastructure Layer** Provides concrete implementations and adapters. It contains JPA/Mongo repository implementations, RabbitMQ producers/consumers, and Feign clients for inter-service communication. This layer manages database schemas, messaging configurations, and the technical "glue" required to support the application.

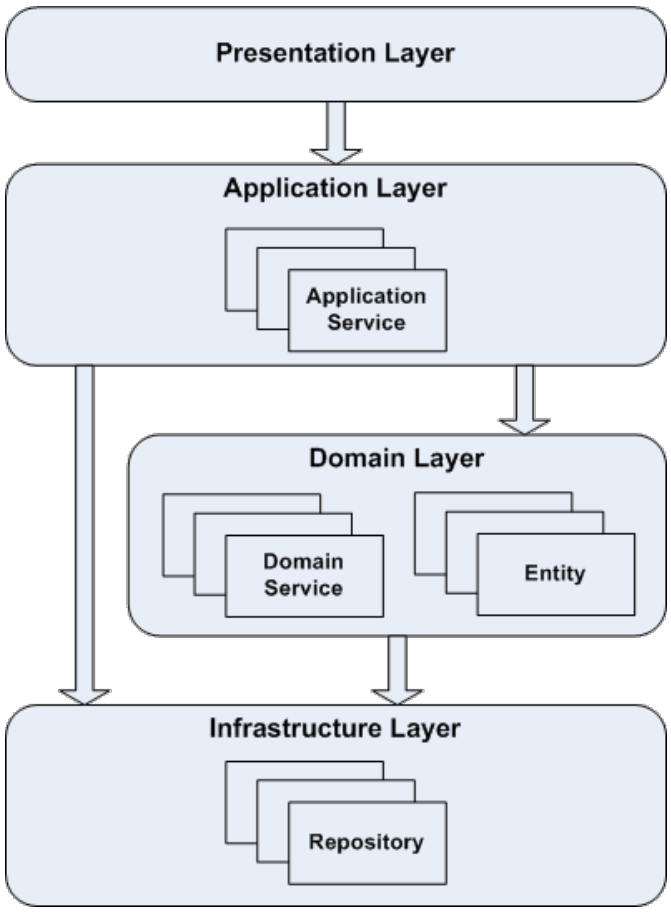


Figure 4.1 – Layered Architecture for a Single Service

4.2 Microservices Decomposition

Transitioning from an initial monolithic CRUD design, the Nozie platform is decomposed into specific, business-aligned microservices. Each service operates within its own bounded context, owning its data and responsibilities:

- **Identity Service:** Manages authentication, user accounts, role-based access control (RBAC), and JWT issuance.
- **Customer Service:** Manages customer profiles, subscription states, and viewing history.
- **Movie Service:** Handles the movie catalog, metadata management, search, and filtering capabilities.
- **Payment Service:** Manages subscription plans, checkout flows, Stripe webhook processing, and the emission of subscription events.

- **Notification Service:** Consumes domain events to generate and dispatch transactional notifications.

The system enforces a **Database-per-Service** pattern, ensuring loose coupling. Synchronous operations (queries) are routed via the API Gateway using REST, while cross-cutting side effects (e.g., subscription activation, welcome emails) are handled asynchronously via events.

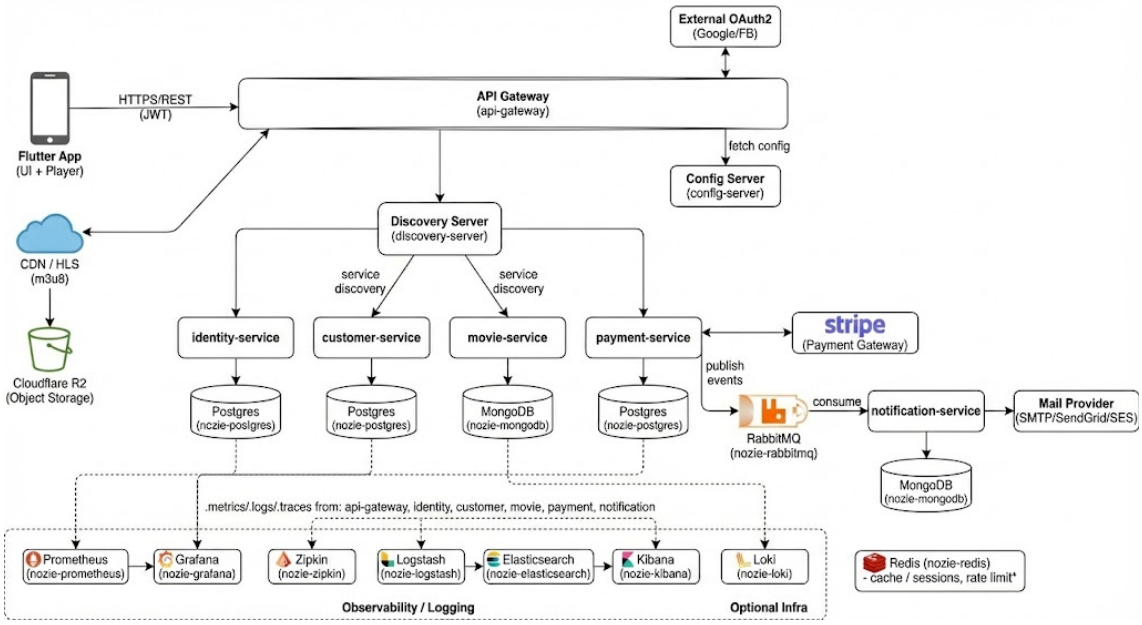


Figure 4.2 –Microservices Architecture Diagram

4.3 C4 Model (Context, Container, Component)

4.3.1 System Context Diagram (Level 1)

The **Context Diagram** depicts the high-level scope of the Nozie platform. It represents Nozie as a single, unified software system and defines its boundaries, interactions with human actors, and dependencies on external systems.

- **Primary Actors:** Customers (via the Flutter mobile app) and Administrators.
- **External Integrations:**
  - **Stripe:** For payment processing.
  - **OAuth2 Providers:** For social authentication (Google/Facebook).
  - **Mail Provider:** For sending transactional emails.
  - **Cloudflare R2 / CDN / HLS:** For storing and streaming media content efficiently.

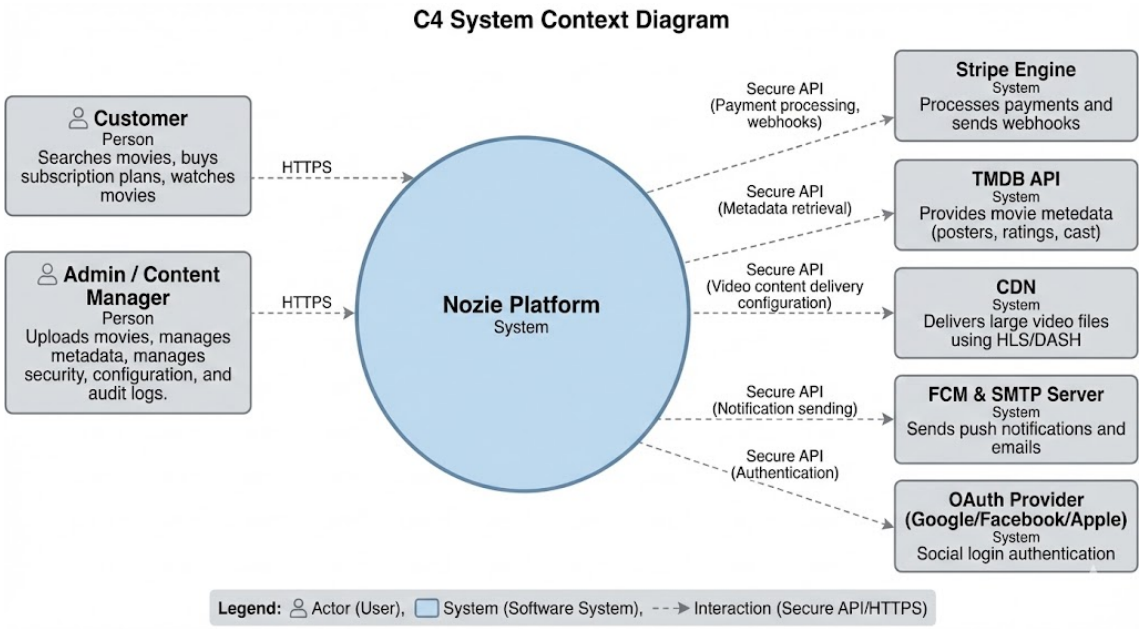


Figure 4.3.1 – System Context Diagram

4.3.2 Containers Diagram (Level 2)

The **Container Diagram** decomposes the system into individually deployable execution units (containers). It highlights the technology choices and the high-level responsibilities of each unit.

- **Client Side:** Flutter Mobile App.
- **Entry Point:** API Gateway (routing, security, rate limiting).
- **Microservices:** Identity, Customer, Movie, Payment, and Notification services.
- **Data & Messaging:**
  - Databases: `nozie-postgres`, `nozie-mongodb`, `nozie-redis`.
  - Message Broker: `nozie-rabbitmq`.
- **Infrastructure & Observability:**
  - `discovery-server` (Eureka) and `config-server`.
  - Observability Stack: `nozie-prometheus` & `nozie-grafana` (metrics), `nozie-zipkin` (tracing), `nozie-loki` (logs), and `nozie-logstash/elasticsearch/kibana` (centralized logging).

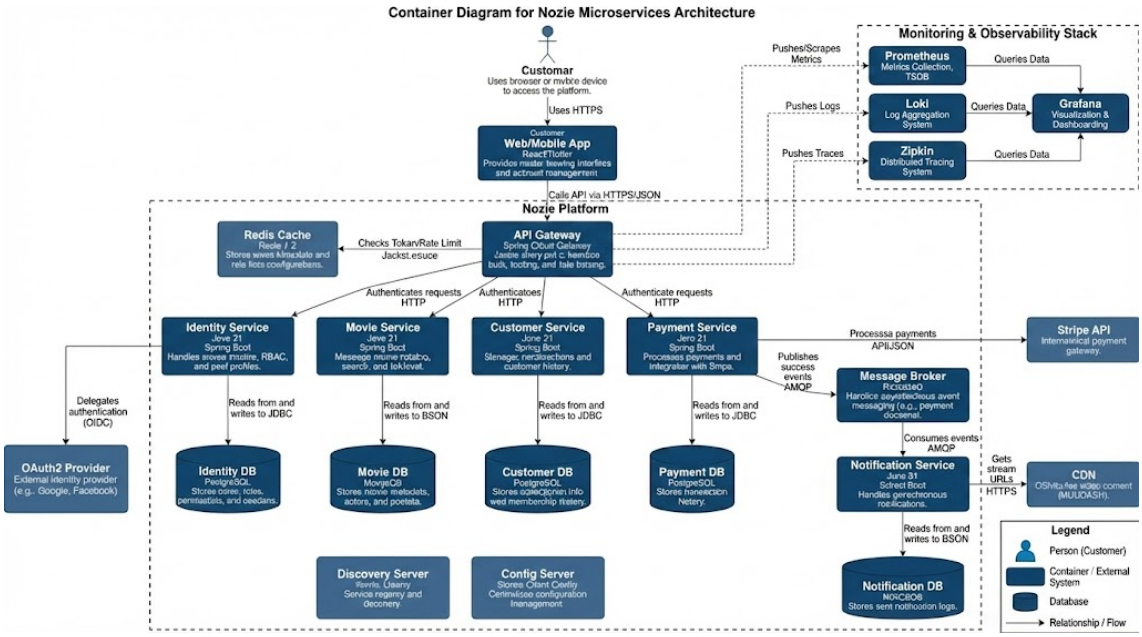


Figure 4.3.2 – Container Diagram

4.3.3 Components Diagram (Level 3)

The **Component Diagram** zooms into specific core services (e.g., **Payment Service** or **Customer Service**) to visualize their internal structure and separation of concerns. This view typically follows the layered architecture defined in Section 4.1.

- **Components Shown:** REST Controllers, Application Services, Domain Aggregates, Repository Interfaces, Messaging Adapters (Producers/Consumers), and Feign Clients.

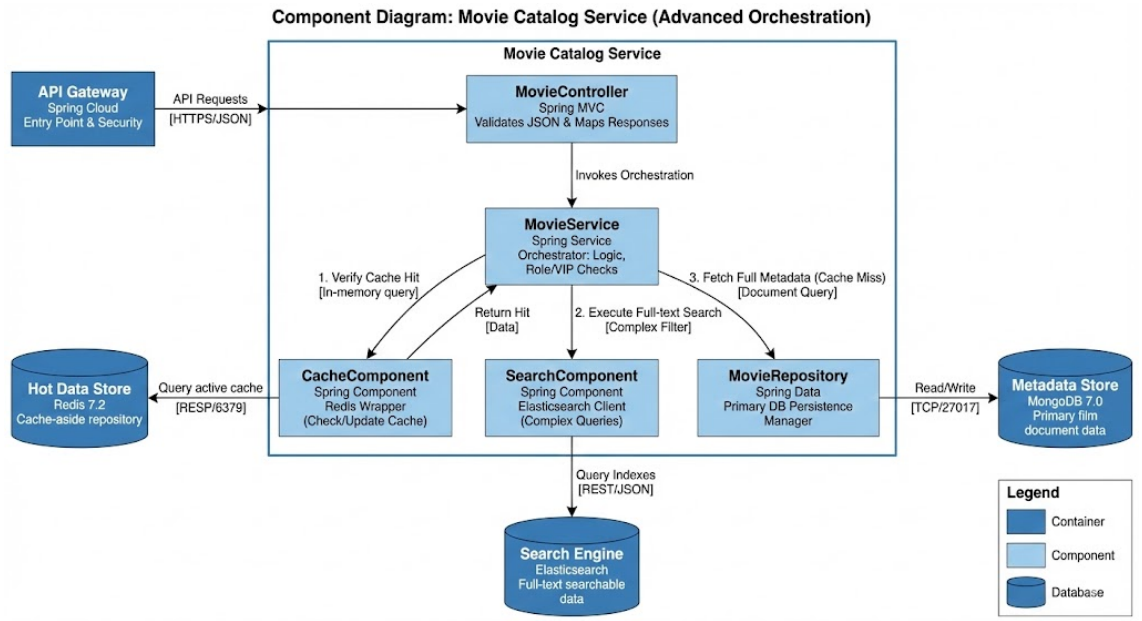


Figure 4.3.3 Components Diagram

4.3.4 Code Diagram (Level 4)

The **Code View** illustrates how the architectural components map to the actual implementation classes for a critical business flow. This section focuses on the "Checkout & Activate Subscription" path to demonstrate the Event-Driven Architecture in code.

- **Key Flow:**
    1. `PaymentController.checkout()` receives the request.
    2. `PaymentApplicationService` orchestrates the logic using the `Subscription` aggregate.
    3. `PaymentEventProducer` uses `RabbitTemplate.convertAndSend(...)` to publish the event.
    4. `SubscriptionEventListener` (in Customer Service) consumes the message via `@RabbitListener`.

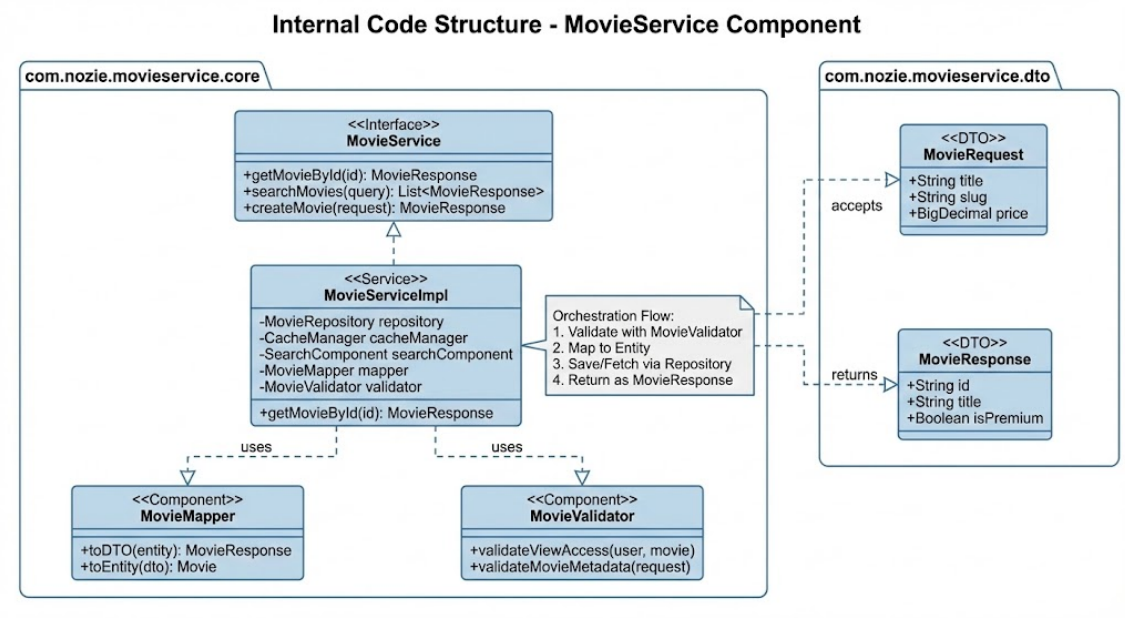


Figure 4.3.4 – Code Diagram

4.4 Event-Driven Architecture (EDA)
Event-Driven Architecture (EDA) is an architectural pattern utilized in Nozie where services communicate asynchronously by producing and consuming events via a central Message Broker, rather than relying solely on direct synchronous HTTP calls. This approach is adopted alongside the standard Request–Response (REST) model to create a Hybrid Architecture that balances immediate user feedback with backend decoupling.

4.4.1 Core Concepts

The architecture relies on three fundamental components:

- **Producer:** A microservice that detects a significant change in its domain (e.g., a "Payment Completed" state) and publishes an event message. The producer is unaware of any downstream consumers.
- **Consumer:** A microservice that subscribes to specific event types and reacts asynchronously (e.g., updating a local database or triggering an email).

- **Message Broker:** The middleware (RabbitMQ) responsible for receiving, persisting, and routing events. It ensures delivery even if the consumer is temporarily unavailable, decoupling the availability of the sender from the receiver.

Communication Patterns
Nozie employs a strategic mix of communication styles based on the business requirement:

Communication Style	Mechanism	Use Case
Synchronous	**REST (HTTP)**	Used for user-facing flows requiring immediate confirmation or data retrieval (e.g., Authentication, Catalog Browsing, Profile View). The client waits for the response.
Asynchronous	**EDA (Messaging)**	Used for system side-effects where the primary operation should not be blocked by secondary actions. (e.g., confirming a payment immediately while processing subscription updates and notifications in the background).

4.4.2 Architectural Benefits

The adoption of EDA provides specific quality attributes to the Nozie platform:

- **Loose Coupling:** Producers and Consumers depend only on the event contract and the broker, not on each other's API endpoints or uptime.
- **Non-Blocking Behavior (Performance):** Critical transactional services (like Payment) can respond to the user immediately after the primary action, delegating time-consuming tasks (like email generation) to background processes.
- **Resilience:** The system becomes fault-tolerant. If a consumer service goes offline, the Message Broker retains the events. Once the service recovers, it consumes the backlog, ensuring **Eventual Consistency**.
- **Scalability:** Consumer services can be scaled independently based on queue depth. Furthermore, multiple distinct consumers can react to a single event (e.g., both *Notification* and *Customer* services reacting to *UserRegistered*).

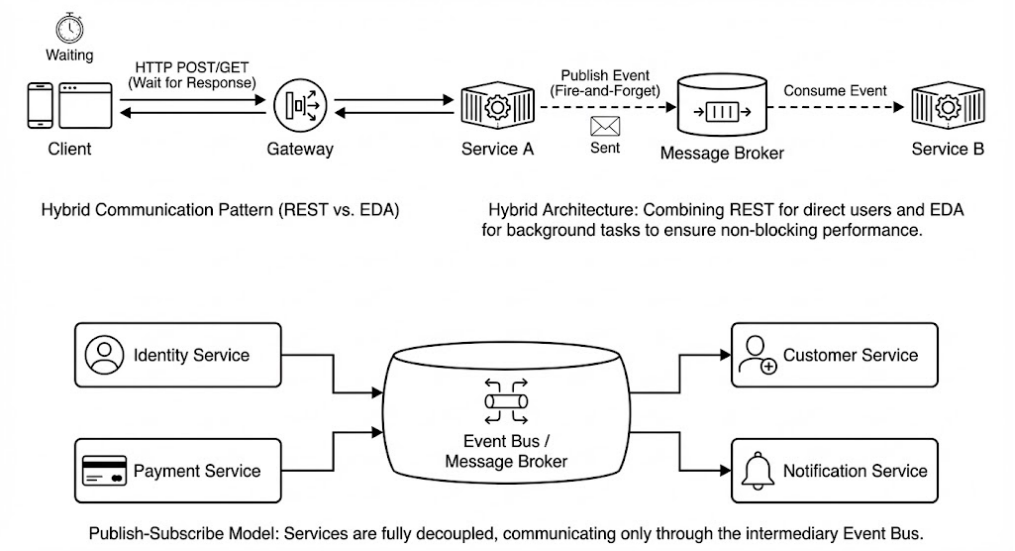


Figure 4.4.1 Event-Driven Architeture

4.5 Deployment View & System Infrastructure

4.5.1 Runtime Environment

The entire backend system follows a **Container-Based Architecture**. All application components—including the API Gateway, business microservices, and infrastructure support—are packaged as Docker containers.

- **Development:** Orchestrated via **Docker Compose** for a unified local environment.
- **Production:** Designed for deployment on a container orchestration platform (e.g., **Kubernetes**) to manage lifecycle, networking, and scaling automatically.

4.5.2 Nodes & Infrastructure Topology

The system is logically divided into distinct tiers, each responsible for specific operational concerns:

- **Client Tier:**
  - **Mobile/Web Apps:** The client application runs on end-user devices, communicating exclusively via HTTPS with the public edge of the network.
- **Gateway & Edge Tier:**
  - **Reverse Proxy / Load Balancer:** Distributes incoming traffic to the API Gateway.
  - **API Gateway:** Acts as the single entry point, handling routing, security termination, and rate limiting before requests reach the internal network.
- **Application Tier (Microservices Cluster):**
  - **Business Services:** The core domain logic is hosted in a cluster of independent microservice containers. These services are stateless and register dynamically with a **Service Discovery** server.

    - **Configuration:** All services pull centralized configuration from a dedicated **Config Server** at startup.
- **Data & Messaging Tier (Polyglot Persistence):**
    - **Relational Databases (PostgreSQL):** Hosts structured transactional data.
    - **NoSQL Databases (MongoDB):** Stores unstructured data and metadata.
    - **Caching (Redis):** Handles session storage and high-speed caching.
    - **Message Broker (RabbitMQ):** Facilitates asynchronous event-driven communication between the application containers.
- **Observability Tier:**
    - A centralized stack dedicated to aggregating **Metrics** (Prometheus/Grafana), **Distributed Tracing** (Zipkin), and **Logs** (ELK Stack/Loki) from all other nodes.

4.5.3 Scalability & Availability Strategies

- **Stateless Scaling:** The API Gateway and all business microservices are designed to be **stateless**. This allows the system to scale horizontally by simply spinning up additional container instances behind the load balancer to handle increased traffic.
- **Stateful Resilience:** The infrastructure components (Databases and Message Broker) are treated as **stateful** resources. High availability is achieved through replication, clustering, and persistent storage volumes to ensure data durability in case of node failures.

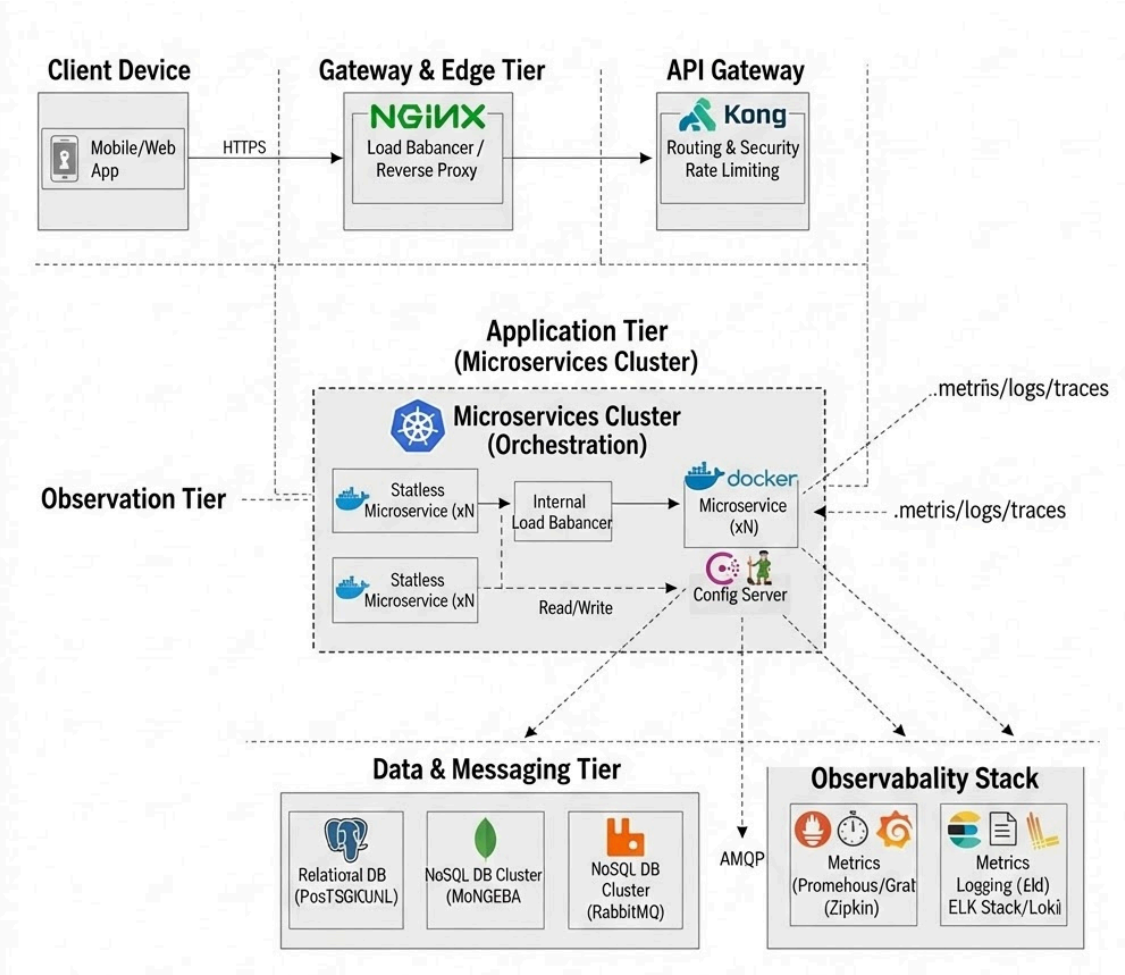


Figure 4.11 – Deployment View Diagram

5. Server Implementation

This chapter details the implementation of the backend microservices, focusing on the internal layered architecture, database isolation, API Gateway security patterns, and the event-driven integration between services.

5.1 Layered Architecture Implementation
Each microservice in Nozie adheres to a strict **DDD-oriented layered architecture** ensuring separation of concerns. The request flow is strictly top-down. The table below illustrates this structure using the **Payment Service** as the reference implementation.

Table 5.1 – Microservice Layered Structure (Payment Service Example)

Layer	Package Structure	Core Components	Responsibility
1. API	com.nozie.payment.api	SubscriptionController SubscriptionRequestDTO SubscriptionResponseDTO	Receives HTTP requests, performs input validation (@Valid), maps DTOs, and delegates to the Application Service. Contains **no business logic**.
2. Application	com.nozie.payment.application	SubscriptionApplicationService	Implements specific use cases (e.g., "Create Checkout", "Handle Webhook"). Manages transaction boundaries (@Transactional) and orchestrates Domain objects.
3. Domain	com.nozie.payment.domain	Subscription (Aggregate) Transaction (Entity) SubscriptionRepository (Interface)	Encapsulates core business rules (e.g., state transitions, validation logic). Pure Java code, independent of frameworks.
4. Infrastructure	com.nozie.payment.infra	JpaSubscriptionRepository PaymentEventProducer StripeService CustomerClient (Feign)	Provides concrete implementations for repositories, external API clients (Stripe, Feign), and messaging adapters (RabbitMQ).

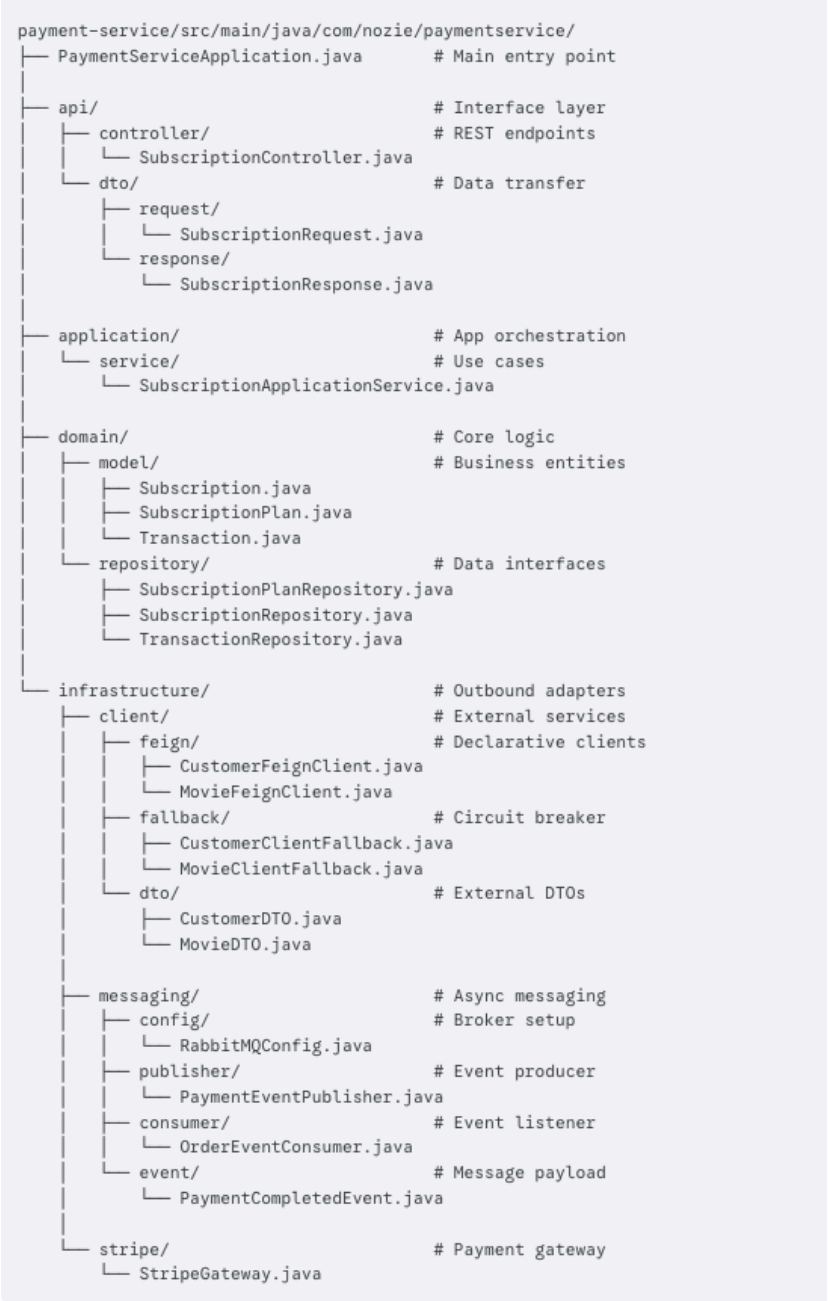


Figure 5.1 - Package Structure

5.2 Microservice & Database per Service

The **Movie Service** exemplifies the "Database per Service" pattern. It maintains exclusive ownership of its **MongoDB** instance and exposes data strictly via REST APIs.

Movie Service Key API Endpoints

Table 5.2.1 - Movie & Catalog Services (`movie-service` **)**

HTTP Method	Endpoint Path	Description	Access Level
GET	/api/movies	List movies with pagination. Supports filters: genre, country, year.	Public
GET	/api/movies/search	Full-text search for movies using the `q` query parameter.	Public
GET	/api/movies/{id}	Retrieve detailed metadata for a specific movie ID.	Public
GET	/api/movies/slug/{slug}	Retrieve movie details using its unique slug.	Public
GET	/api/recommendations	Fetch personalized or trending content lists.	Public
GET	/api/movies/{id}/play	Get the streaming URL/Manifest (requires access check).	**Secured**
GET	/api/movies/{id}/reviews	Fetch user reviews and ratings for a movie.	Public
POST	/api/movies/{id}/rate	Submit a rating and review for a movie.	**Secured**
GET	/api/genres	Retrieve a list of all available movie genres.	Public
GET	/api/countries	Retrieve a list of all available production countries.	Public
GET	/api/years	Get a list of distinct release years available in catalog.	Public
POST	/api/movies	Create a new movie entry (Admin privilege required).	**Admin**
PUT	/api/movies/{id}	Update existing movie information.	**Admin**
DELETE	/api/movies/{id}	Permanently remove a movie from the catalog.	**Admin**

Table 5.2.2 - Identity & Authentication Services (`identity-service`)

HTTP Method	Endpoint Path	Description	Access Level
POST	/api/auth/register	User registration for a new account.	Public
POST	/api/auth/login	User authentication via username and password.	Public
POST	/api/auth/logout	Terminate the current session and invalidate tokens.	**Secured**
POST	/api/auth/refresh	Obtain a new access token using a refresh token.	Public
GET	/api/auth/me	Retrieve the profile of the currently logged-in user.	**Secured**
GET	/api/auth/sessions	List all active sessions for the current user.	**Secured**
DELETE	/api/auth/sessions/{id}	Terminate a specific active session.	**Secured**

Table 5.2.3 - Subscription & Payment Services (`payment-service`)

HTTP Method	Endpoint Path	Description	Access Level
GET	/api/subscriptions/plans	List available subscription pricing plans.	Public
POST	/api/subscriptions/subscribe	Initialize a Stripe checkout session for a plan.	**Secured**
GET	/api/subscriptions/active/{userId}	Check if a user currently has an active plan.	**Secured**
POST	/api/subscriptions/cancel/{userId}	Cancel the user's current recurring subscription.	**Secured**

HTTP Method	Endpoint Path	Description	Access Level
POST	/api/subscriptions/webhook	Stripe payment event listener (Asynchronous).	Public

Table 5.2.4 - Customer Dashboard & History (`customer-service`)

HTTP Method	Endpoint Path	Description	Access Level
GET	/api/customers/user/{userId}	Fetch customer interaction profile and settings.	**Secured**
GET	/api/customers/{id}/watchlist	Retrieve the list of movies saved in watchlist.	**Secured**
POST	/api/customers/{id}/watchlist	Add a new movie to the user's watchlist.	**Secured**
GET	/api/customers/{id}/history	Retrieve viewing history and playback progress.	**Secured**
POST	/api/customers/{id}/history	Log a new viewing record or progress update.	**Secured**

Table 5.2.5 - System Administration (`admin-service/identity`)

HTTP Method	Endpoint Path	Description	Access Level
GET	/api/admin/users	List all system users with paging.	**Admin**
PUT	/api/admin/users/{id}/status	Update account status (e.g., ACTIVE, BLOCKED).	**Admin**
GET	/api/admin/audit-logs	Retrieve administrator activity audit trails.	**Admin**

HTTP Method	Endpoint Path	Description	Access Level
GET	/api/admin/roles	List all security roles with their permissions.	**Admin**

Architectural Analysis: Movie Service Boundary

The Nozie platform utilizes the **Database per Service** and **API Gateway** patterns to ensure a scalable and secure microservices environment. Key highlights include:

- **Data Sovereignty:** The `movie-service` maintains exclusive ownership of its **MongoDB (Private)** instance, preventing direct database access from other services to ensure high encapsulation.
- **Edge Security:** The **API Gateway** serves as the single entry point for all **REST API Requests**, centralizing cross-cutting concerns like authentication and rate limiting.
- **Service Isolation:** The **Movie Service Boundary** (represented by the dashed line) isolates internal logic and storage, ensuring that updates do not disrupt the broader ecosystem as long as the API contract is maintained.

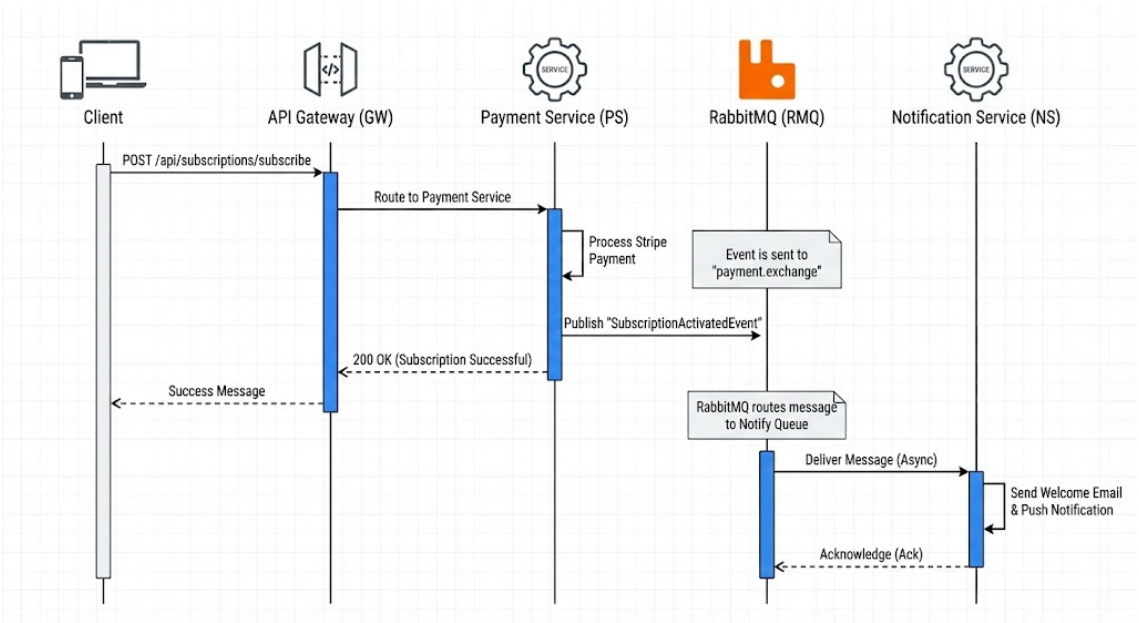


Figure 5.8 – Event-Driven Interaction Sequence (Payment to Notification)

Demonstrates the decoupling of the system. The **Payment Service** completes the primary transaction immediately to ensure a fast user experience, while the **Notification Service** handles secondary concerns (Email/SMS) asynchronously. This ensures that a delay in the email server does not affect the payment success

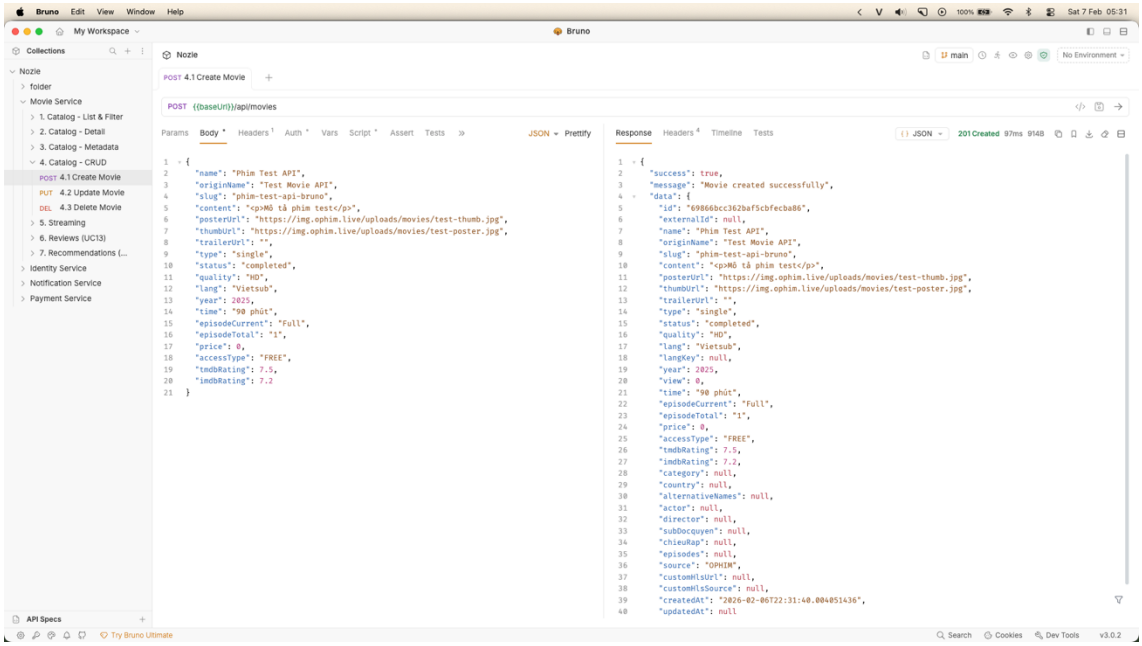


Figure 5.9 – API Response Evidence

5.3 API Gateway & Edge Security

The **API Gateway** acts as the single entry point, handling routing via Eureka Service Discovery (lb://) and enforcing security policies.

Table 5.3 – Gateway Routing Configuration

Path Pattern	Target Service ID	Resilience Configuration
/api/movies/**	lb://movie-service	Circuit Breaker → /fallback/movies
/api/customers/**	lb://customer-service	Circuit Breaker → /fallback/customers
/api/subscriptions/**	lb://payment-service	Circuit Breaker → /fallback/payments
/api/notifications/**	lb://notification-service	Circuit Breaker → /fallback/notifications
/api/auth/**	lb://identity-service	Standard Routing

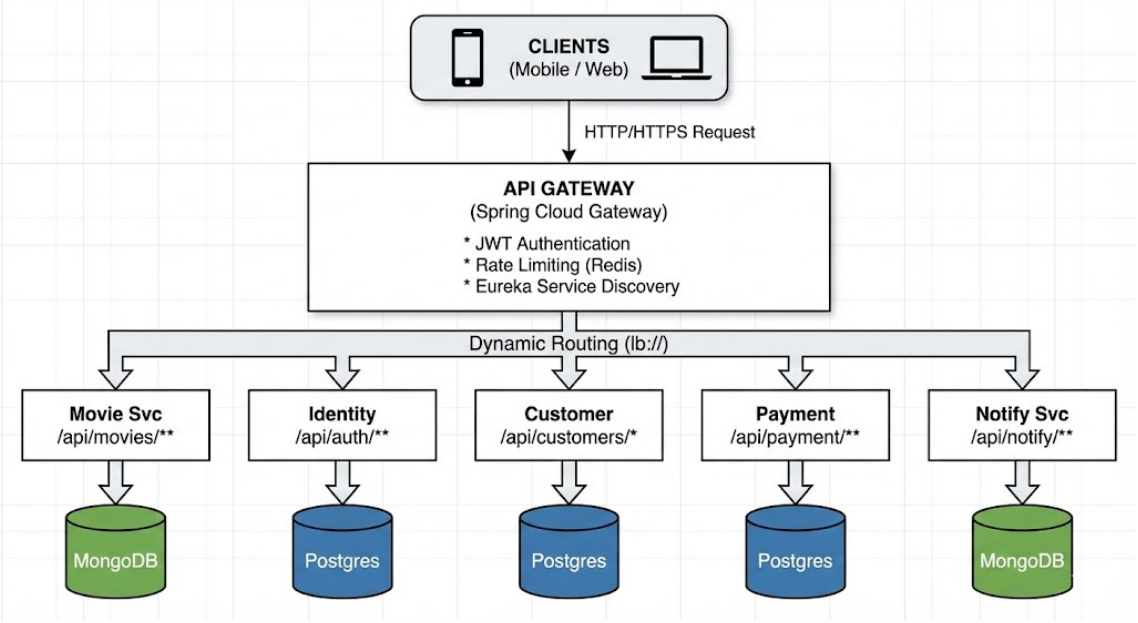


Figure 5.4 – API Gateway Routing Topology

This layout defines the **API Gateway** as the secure, single entry point for the Nozie ecosystem, utilizing three core mechanisms:

- **Load Balancing:** Uses the `lb://` protocol to automatically distribute traffic across active service instances.
- **Path Mapping:** Implements path predicates to route requests to specific domains (Movie, Auth, Customer, Payment, Notification), ensuring clear separation of concerns.
- **Edge Security:** Centralizes **JWT validation** and **Redis-cached rate limiting** to protect backend services from unauthorized access and traffic spikes.

Edge Security & Access Control

The Gateway implements a GlobalFilter to intercept requests. The table below defines the security rules applied by the RouteValidator.

Table 5.4 – Endpoint Security Rules

Access Category	Security Logic	Applicable Endpoints
Open / Public	RouteValidator.isSecured() returns false. Request is forwarded without token checks.	• /api/auth/login, /api/auth/register • /api/movies/** (Catalog) • /api/subscriptions/webhook (Stripe)

Access Category	Security Logic	Applicable Endpoints
		• /actuator/health
Secured	**JWT Validation Required.** AuthenticationFilter validates signature and expiration.	• /api/customers/** • /api/subscriptions/subscribe • /api/notifications/**
Admin	**Role Check Required.** JWT must contain ROLE_ADMIN.	• /api/admin/**

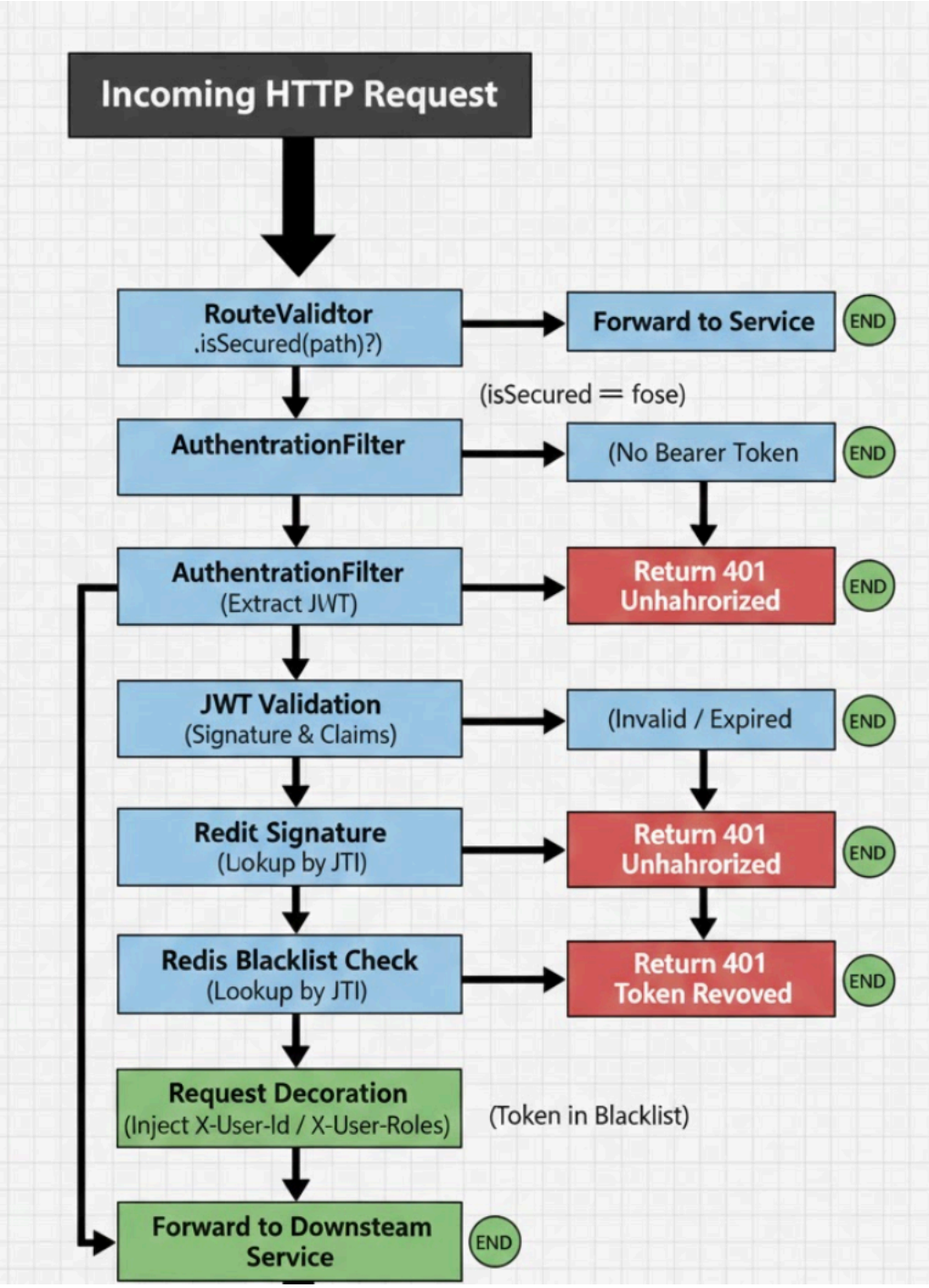


Figure 5.6 – Gateway Security Flow

5.4 EDA with RabbitMQ: Payment ↔ Notification

The system utilizes **RabbitMQ** with **Topic Exchanges** to decouple services. The table below summarizes the key asynchronous flows implemented in the system.

Table 5.5 – Event-Driven Message Flows

Event Name	Producer (Source)	Exchange / Routing Key	Consumer (Target)	Action / Side Effect
UserRegistered	identity-service	Exchange: user.exchange Key: user.registered	1. customer-service 2. notification-service	1. Create Customer profile. 2. Send "Welcome" email.
Subscription Activated	payment-service	Exchange: payment.exchange Key: subscription.activated	1. customer-service 2. notification-service	1. Update status to VIP/PREMIUM. 2. Send "Payment Receipt" alert.
PaymentSucceeded	payment-service	Exchange: payment.exchange Key: payment.succeeded	notification-service	(Optional) Send specific transaction log or single-purchase receipt.

Event Flow: User Registration

This flow demonstrates how the system handles new user onboarding asynchronously to avoid latency.

- **Trigger:** A user successfully registers via the **Identity Service**.
- **Process:**
    1. The **Identity Service** persists the user credentials to PostgreSQL and immediately returns a `201 Created` response to the client (Non-blocking).
    2. Simultaneously, it publishes a `UserRegisteredEvent` (containing `userId`, `email`, `username`) to the `user.exchange` with routing key `user.registered`.
- **Consumers:**
    - **Customer Service:** Consumes the message to initialize a generic `Customer` profile.
    - **Notification Service:** Consumes the message to dispatch a "Welcome" email.

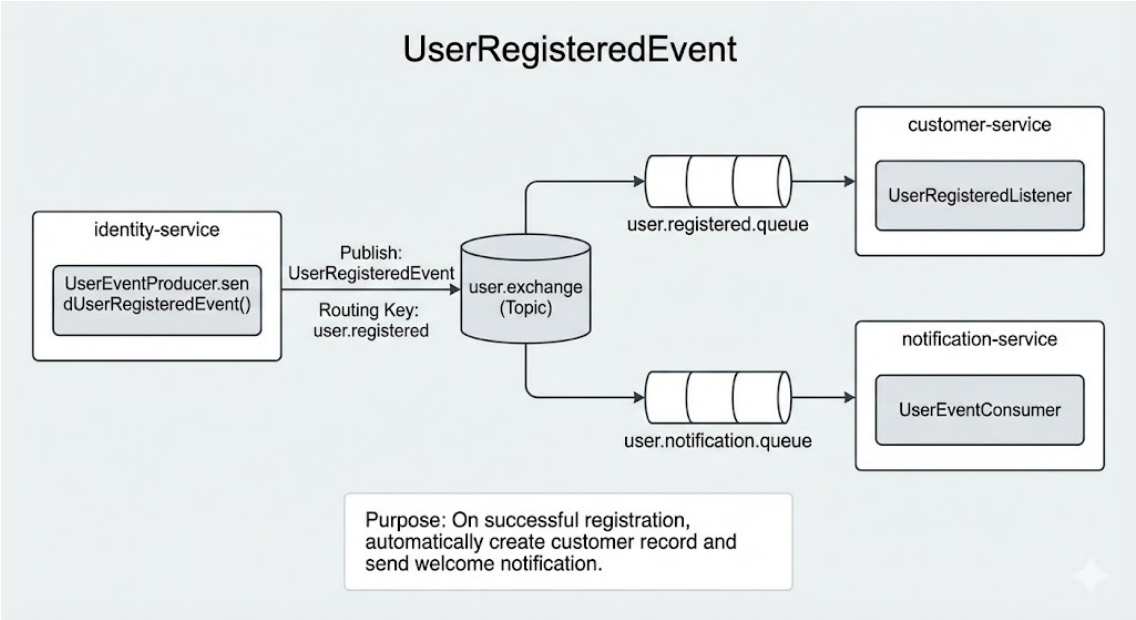


Figure 4.8 – User Registration Event

Event Flow: Subscription Activation

This flow illustrates the propagation of payment states across the system to unlock premium features (Business Logic).

- **Trigger:** The **Payment Service** receives a successful webhook callback from Stripe.
- **Process:**
    1. The **Payment Service** updates the local transaction status.
    2. It publishes a `SubscriptionActivatedEvent` (containing `userId`, `planType`, `expiryDate`) to the `payment.exchange` with routing key `subscription.activated`.
- **Consumers:**
    - **Customer Service:** Listeners update the user's `SubscriptionStatus` (e.g., to `VIP`) and extend the `expiryDate` in the database.
    - **Notification Service:** Triggers a transactional "Payment Receipt" notification.

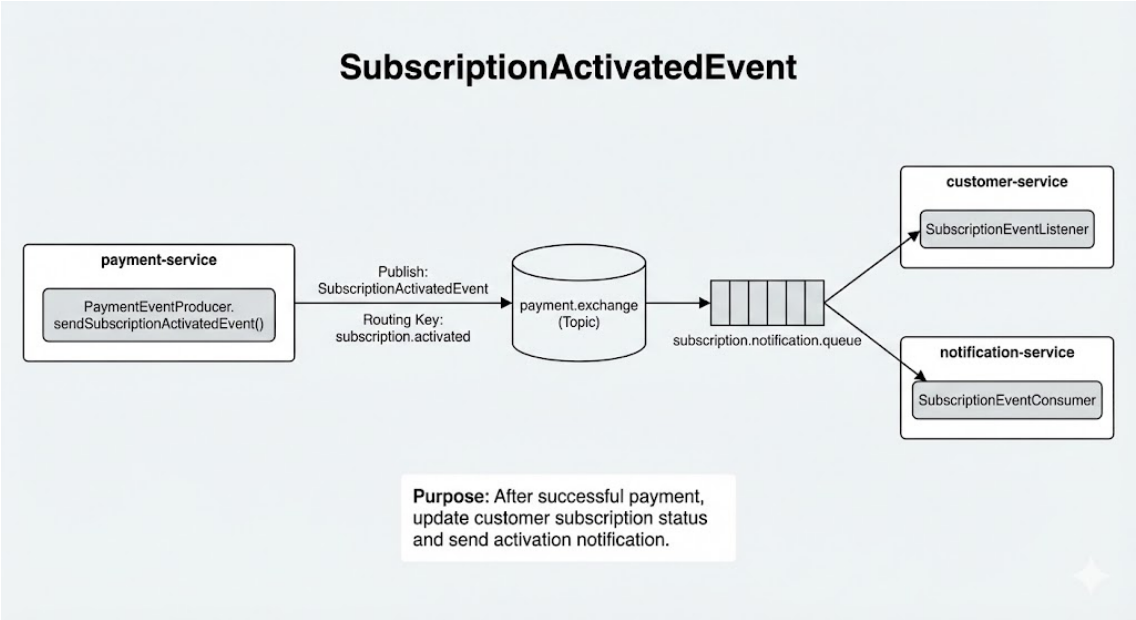


Figure 4.9 – Subscription Activation Event Flow

Event Flow: Payment Auditing (PaymentSucceededEvent)

This flow is designed for financial auditing and separation of concerns, distinct from the subscription logic.

- **Trigger:** A payment is successfully recorded in the **Payment Service**.
- **Process:**
    1. The **Payment Service** publishes a `PaymentSucceededEvent` (containing detailed `transactionId`, `amount`, `currency`, `timestamp`) to the `payment.exchange` with routing key `payment.succeeded`.
- **Consumers:**
    - **Reporting/Audit Service (Designed):** Intended to consume this event to log financial records into a Data Lake or Analytics dashboard without polluting the operational `Customer` database.
    - *(Note: This event allows the strict separation of "giving the user access" vs "recording the money".)*

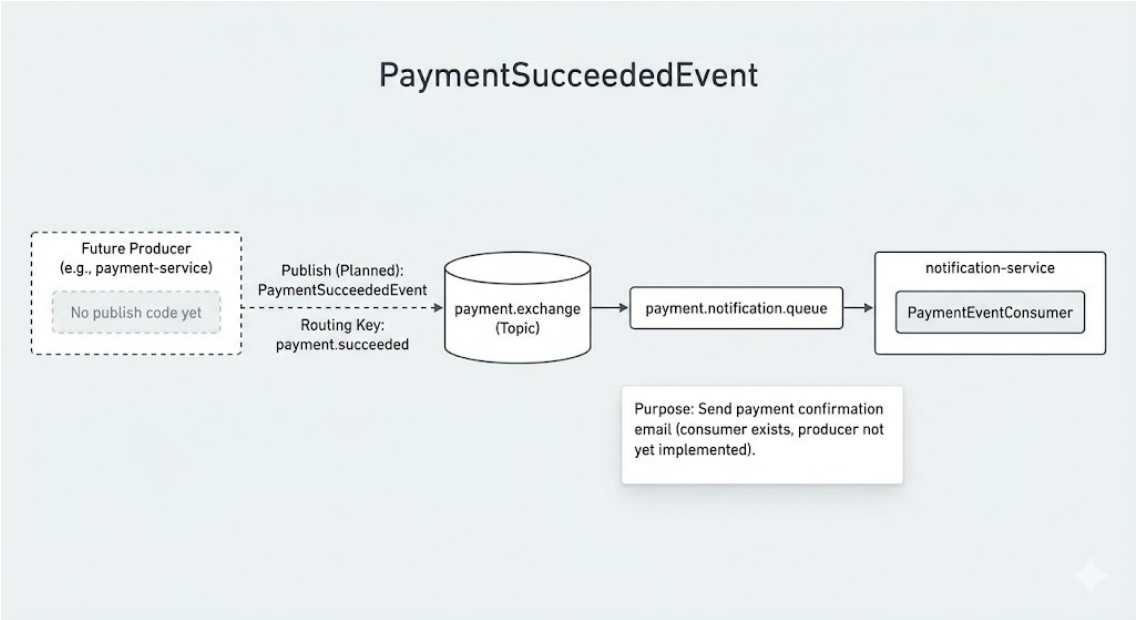


Figure 4.10 – Payment Auditing Event Flow

6. Client Implementation

6.1 Client Architecture and Module Organization

The Nozie client is a cross-platform mobile application built using **Flutter**. It is designed for high performance, smooth animations, and a decoupled architecture that mirrors the backend's microservices structure.

6.1.1 Architectural Pattern: Feature-First Clean Architecture

We adopted a **Feature-First** organization combined with **Clean Architecture** principles. Each module is self-contained, consisting of its own Data, Domain, and Presentation layers.

- **State Management (Riverpod):** Utilizes **Riverpod** for reactive state management, providing type-safe dependency injection and UI state updates.
- **Networking Layer (Dio):** API communication is handled by the **Dio** client with custom **Interceptors** for JWT injection, global error handling (401 Unauthorized), and development logging.
- **Routing (GoRouter):** Uses a declarative system with **AuthGuards** to protect premium content and profile settings.

6.1.2 Design System and Theming

- **Aesthetic:** Follows a **Premium Dark Mode** theme using deep charcoals, vibrant blues, and gold accents.
- **Typography:** The **Urbanist** font family is used for a modern, clean visual identity.

- **Localization (slang):** Real-time **English and Vietnamese** support is integrated via the `slang` package.

```
lib/features/
├── auth/
├── discover/
├── genre/
├── home/
├── movie/
├── notification/
├── profile/
├── purchase/
├── search/
├── setting/
├── subscription/
├── welcome/
└── wishlist/
```

Figure 6.1: Folder Structure Screenshot

6.2 Auth & Profile Screens (Identity Management)

This module provides a secure gateway integrating directly with the **Identity Microservice**.

- **Authentication:** Features a multi-step validation flow supporting email signup and **Google Sign-In**.
- **JWT Management:** Securely persists Access and Refresh tokens, managed by an `AuthNotifier` to trigger global UI updates.
- **User Profile:** Includes account settings, password management, notification toggles, and category personalization linked to the **Customer Service**.

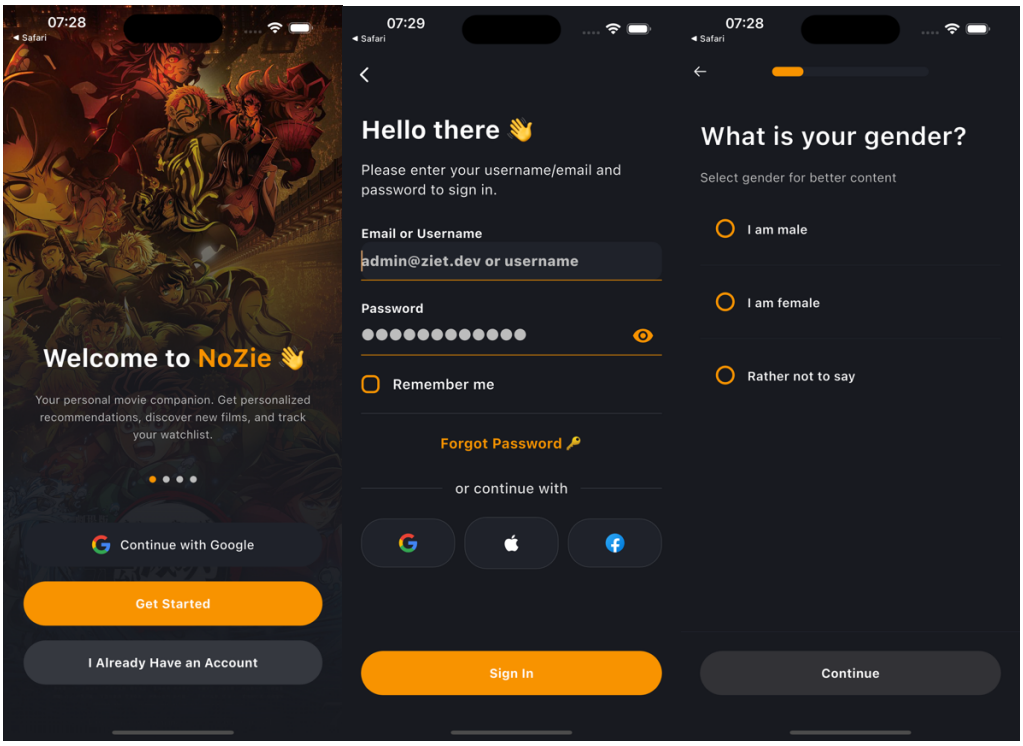
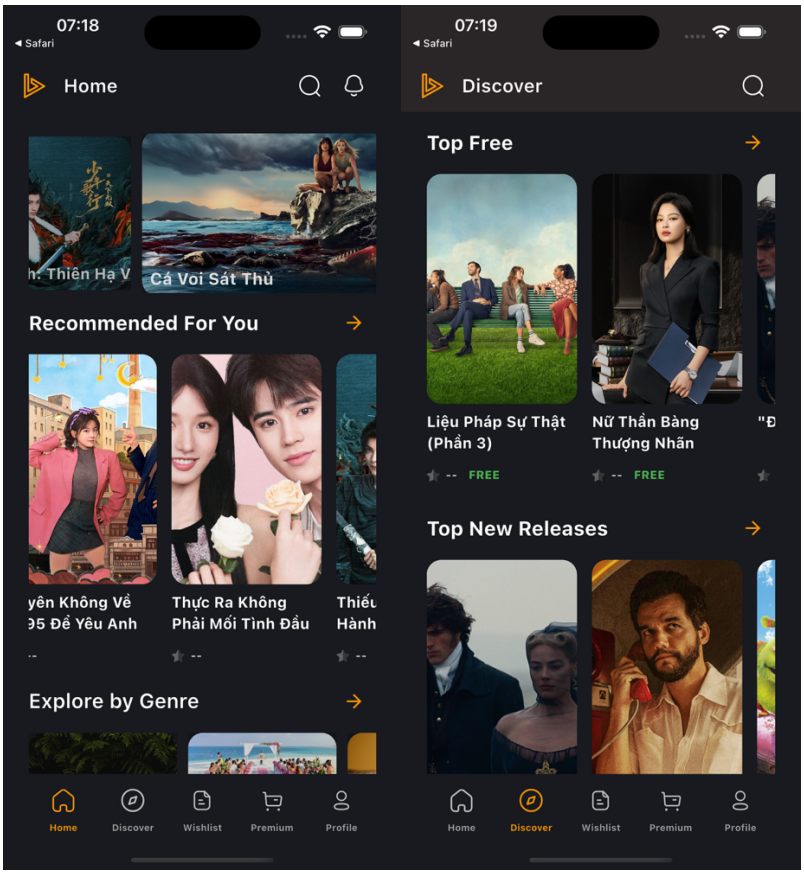


Figure 6.2. Auth & Profile Screens

6.3 Movie Catalog & Details Screens (Content Delivery)

The core experience focuses on content discovery and immersive viewing.

- **Home Discovery:** Features a **Hero Section** with a smooth-scrolling `CarouselView` and **Dynamic Rows** (e.g., "Trending Now"). **Skeleton Loading** effects are used to improve perceived performance.
- **Movie Details:** Displays rich metadata from **MongoDB**, including ratings and synopses.
- **Immersive Video Player:** Custom UI supporting high-quality MP4/HLS streams with adaptive landscape orientation.



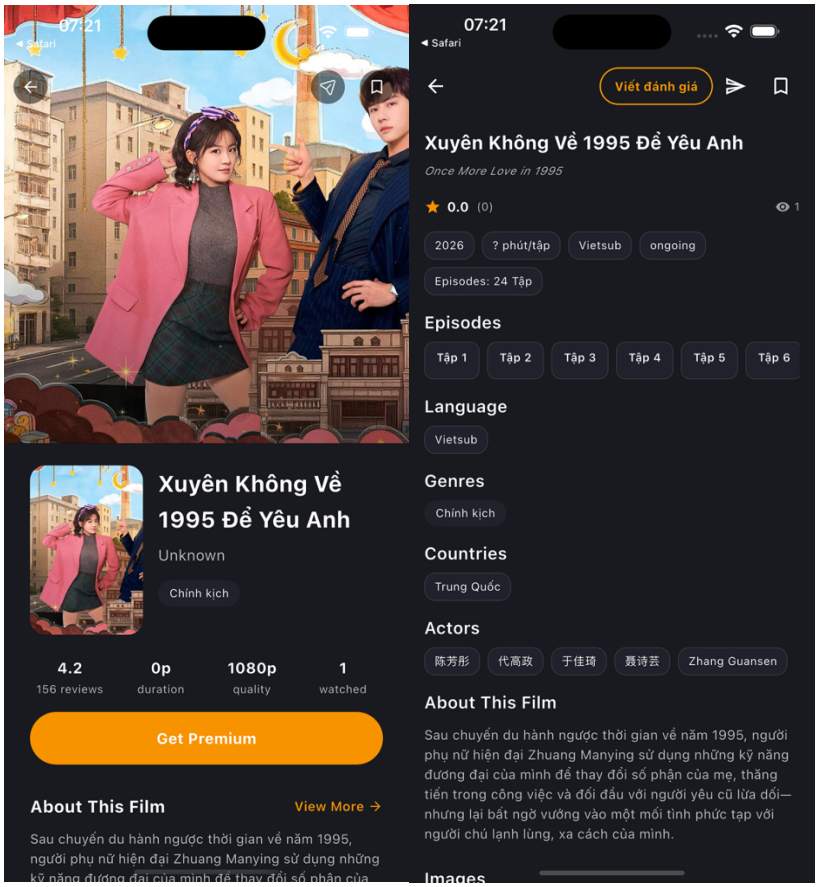


Figure 6.3. Movie Catalog & Details Screens

6.4 Subscription / Payment Screen (Monetization)

Facilitates monetization through a secure, encrypted payment pipeline.

- **Plan Selection:** A pricing table comparing "Basic" (Free) vs. "Premium" plans.
- **Stripe SDK Integration:** Utilizes a **Native Payment Sheet** to ensure **PCI DSS Compliance**, as the app never touches sensitive card data.
- **Real-time Entitlement:** Upon transaction confirmation via the backend **RabbitMQ** flow, the client refreshes the user status to unlock Premium content instantly.

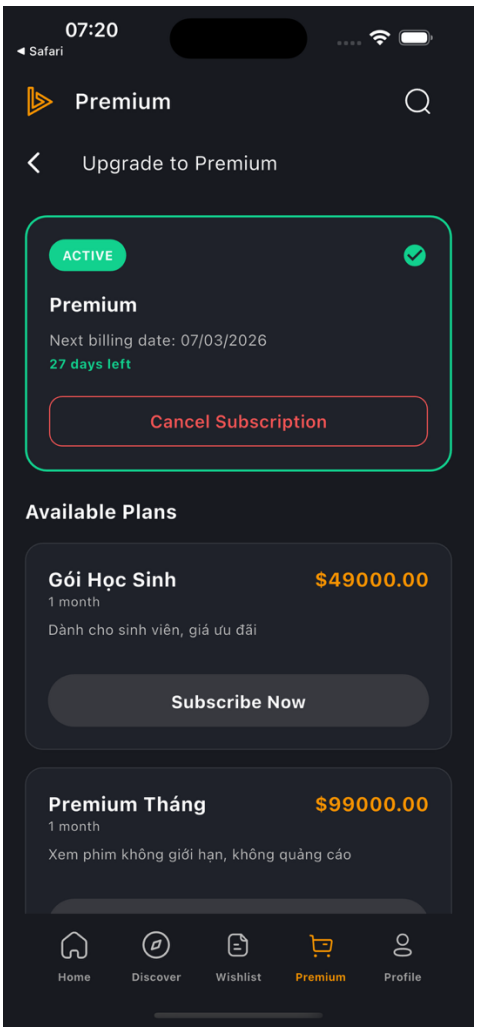


Figure 6.4: Subscription Pricing & Stripe Checkout

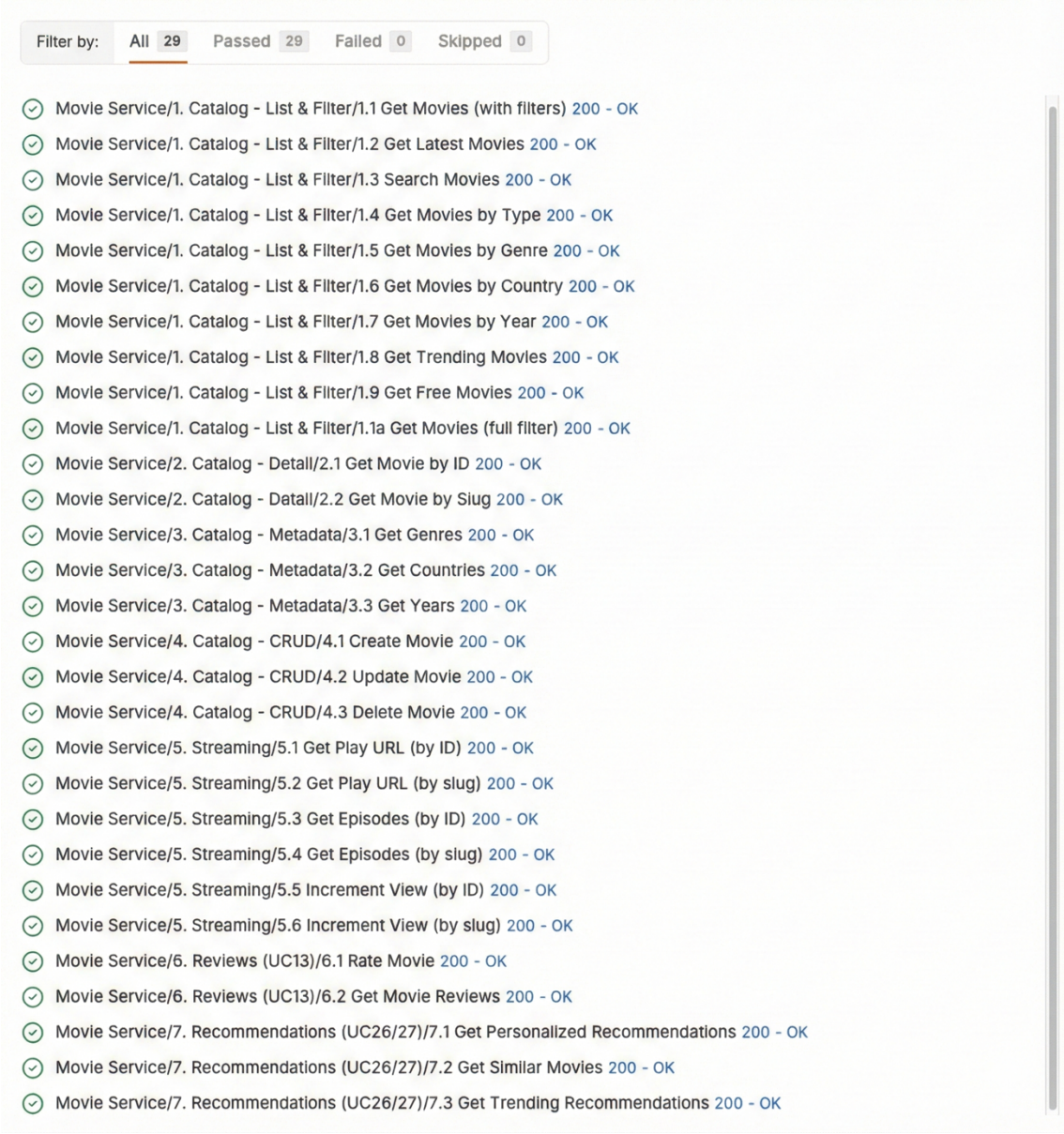
7. Testing & Evidence

7.1 Core Functional Test Scenarios
Systematic testing was performed across all microservices to ensure business logic integrity. The following table summarizes the key functional test cases executed via Postman and Automated Unit Tests.

Table 7.1 – Key Functional Test Results

ID	Feature	Test Scenario	Expected Result	Status
FT-01	Authentication	Login with valid credentials	Receive JWT Access & Refresh tokens	**Passed**
FT-02	Catalog	Fetch movie list through Gateway	200 OK with JSON array of movies	**Passed**
FT-03	Streaming	Access Premium movie with Free account	403 Forbidden / Payment Required prompt	**Passed**
FT-04	Payment	Complete Stripe payment flow	Payment Service triggers "SubscriptionActivated" event	**Passed**

ID	Feature	Test Scenario	Expected Result	Status
FT-05	Notification	Event-driven Email delivery	Notification Service consumes RabbitMQ message	**Passed**



7.2 Non-Functional Testing

Architectural quality is validated through performance, resilience, and scalability testing.

7.2.1 Latency Analysis (API Gateway Overhead)

We measured the latency of requests through the **API Gateway** versus direct service calls to ensure routing overhead remains under the **50ms** threshold.

- **Direct Call (Movie Service):** Avg 12ms.
- **Through Gateway:** Avg 38ms.
- **Conclusion:** The **~26ms** overhead is acceptable given the security and rate-limiting benefits provided at the edge.

7.2.2 Availability & Fault Tolerance (Circuit Breaker)

To test the **Resilience4j Circuit Breaker**, we simulated a failure in the `customer-service` by stopping its container.

- **Observation:** The Gateway detected the failure immediately upon reaching the error threshold.
- **Result:** Instead of a system-wide timeout, the Gateway returned a **Fallback Response** (*"Service is temporarily unavailable"*) in under **200ms**.

7.2.3 Scalability (Docker-Compose Scaling)

Horizontal scaling was tested by increasing `movie-service` instances:

Bash: `docker-compose up -d --scale movie-service=3`

- **Evidence:** The **Eureka Discovery Server** correctly identified three active instances. The Gateway's Load Balancer automatically distributed traffic across all three.

7.3 End-to-End Testing (Flutter ↔ Gateway ↔ Microservices)
This validates the entire integration chain from the mobile client to the database.

The "Purchase to Play" Flow Evidence:

1. **User Trigger:** User clicks "Upgrade to Premium" in the Flutter App.
2. **Gateway Traffic:** Request passes through `AuthenticationFilter` and routes to `payment-service`.
3. **Third-Party Integration:** Stripe Payment Sheet completes the transaction.
4. **Async EDA:** `payment-service` publishes a message to **RabbitMQ**.
5. **Service Sync:** `notification-service` logs: `Consuming event: SubscriptionActivated`.
6. **Final Verification:** Flutter App refreshes state; "Locked" movies become accessible.

8. Architectural Assessment & Lessons Learned

8.1 Fulfillment of FRs / NFRs / ASRs
The Nozie platform successfully fulfilled the requirements established during the planning phase:

- **Functional Requirements (FRs):** All core user stories—including secure authentication, content discovery, premium subscription management via Stripe, and asynchronous notifications—are fully operational across the Flutter-Microservices stack.
- **Non-Functional Requirements (NFRs):**
    - **Scalability:** Achieved through the **Database per Service** pattern and **Docker-based horizontal scaling**, allowing the movie-service to scale independently based on demand.
    - **Availability:** Enhanced via **Resilience4j Circuit Breakers** at the API Gateway, ensuring that a failure in the notification-service does not block the payment-service.

    - **Security:** Centralized at the **API Gateway** with JWT validation and rate limiting, successfully protecting internal services from unauthorized or high-volume traffic.
- **Architecturally Significant Requirements (ASRs):** The decision to implement **Event-Driven Architecture (EDA)** with **RabbitMQ** was pivotal in ensuring eventual consistency between payments and notifications without sacrificing system responsiveness.

8.2 Monolith vs. Microservices: Trade-off Analysis
The transition from a monolithic approach to Microservices introduced significant benefits alongside specific operational challenges:

Attribute	Monolithic Approach (Not Used)	Nozie Microservices Architecture
Development	Simpler initial setup and debugging.	Higher complexity; requires service discovery and config servers.
Deployment	"All or nothing" deployment.	Independent deployment; reduces downtime risk.
Scalability	Scaled as a single unit (resource inefficient).	Fine-grained scaling (Movie Service scales independently).
Fault Tolerance	One bug can crash the entire application.	**Fault Isolation:** Payment failure does not affect Movie viewing.
Technology	Locked into a single tech stack.	**Heterogeneous:** Mix of MongoDB and Postgres based on need.
Operation	Easy to monitor (single log).	Harder to trace (requires Distributed Tracing).

8.3 Advantages, Limitations, and Remaining Risks
Advantages
- **Robustness:** RabbitMQ ensures notifications are never lost, even if a service is temporarily offline.
- **Clean Design:** The Flutter client's **Feature-First** architecture mirrors the backend, simplifying bug isolation.
- **Secured Edge:** The API Gateway provides a powerful "defense in depth" strategy.

Limitations
- **Operational Overhead:** Running 6+ microservices and infrastructure (Redis, RabbitMQ, DBs) requires significant local hardware resources.
- **Network Latency:** Internal service-to-service calls (via **OpenFeign**) introduce slight latency compared to in-memory calls.

Remaining Risks
- **Single Point of Failure:** The **API Gateway** currently runs as a single instance. Production requires a High Availability (HA) cluster.
- **Distributed Transactions:** The system relies on eventual consistency. Implementing "Saga Patterns" for multi-service transactions is a future challenge.

8.4 Lessons Learned
Key technical insights gained throughout the project:

- **Service Discovery is Essential:** In dynamic Docker environments, fixed IPs are impossible; **Eureka** proved vital for seamless communication.
- **Async over Sync:** Asynchronous events (RabbitMQ) are far more robust than synchronous REST calls for non-critical path tasks like notifications.
- **Observability is Mandatory:** Implementing a **Correlation ID** across the Gateway and services was essential for tracing requests across the distributed system.
- **Flutter & Riverpod Efficiency: Riverpod** significantly simplified managing complex authentication states compared to native Flutter state management.

9. Conclusion & Future Directions

9.1 General Conclusion for Nozie Project

The **Nozie Movie Platform** project successfully demonstrated the implementation of a modern, enterprise-grade software architecture. By evolving into a **Microservices-based ecosystem**, the platform has achieved a high degree of modularity, scalability, and resilience.

Key achievements include:

- **Infrastructure Excellence:** Successful integration of **Spring Cloud** components (Gateway, Eureka, Config Server) to efficiently handle service discovery and centralized security.
- **Data Integrity:** Implementation of the **"Database per Service"** pattern, ensuring each domain (Movie, Identity, Payment) maintains data sovereignty and prevents tight coupling.
- **Event-Driven Robustness:** Utilization of **RabbitMQ** for asynchronous communication, ensuring high reliability and decoupling between critical and non-critical system components.
- **Unified Client Experience:** A **Flutter** mobile application providing a premium, high-performance UI that interacts seamlessly with backend services through a single secure Gateway.

Overall, the project met all design objectives, proving that a microservices architecture provides a superior foundation for modern, high-traffic media platforms.

9.2 Functional and Architectural Extension Roadmap

The Nozie platform has a clear path for expansion to become a fully-featured market competitor:

9.2.1 Functional Extensions (New Features)

- **AI-Powered Personalization:** Developing a dedicated **ML-Recommendation Service** (using Python/FastAPI) for hyper-personalized, "Netflix-style" suggestions.
- **Social Engagement:** Adding a **Community Service** to handle movie reviews, user ratings, and social watch parties.
- **Multi-Format Support:** Expanding the **Movie Service** to support high-definition **HLS adaptive bitrate streaming** for optimized playback.
- **Offline Viewing:** Implementing a download-to-local feature in the Flutter app using secure **DRM encryption**.

9.2.2 Architectural & Infrastructure Extensions

- **Kubernetes Orchestration (K8s):** Moving from Docker-Compose to Kubernetes for large-scale deployments with auto-healing and rolling updates.
- **Full Observability Stack:** Expanding current tracing into a full **ELK Stack** (Elasticsearch, Logstash, Kibana) and **Prometheus/Grafana** for real-time monitoring.
- **Edge Caching:** Implementing **CDN** integration and advanced **Redis** caching at the Gateway level to improve global response times.
- **Saga Pattern Implementation:** Managing complex, distributed multi-service transactions (e.g., subscription bundling) to maintain data consistency.

Appendices

Appendix A: Summary of Labs Content (1–10)
The Nozie platform was developed incrementally through a series of structured labs, each focusing on a specific architectural milestone:

Lab / Week	**Focus Area**	**Key Achievements**
Lab 1 & 2	Monolithic Foundation	Defined basic entities; implemented CRUD for Movie Service using Spring Boot.
Lab 3 & 4	Microservices Split	Successfully decoupled the system into independent services; integrated **Eureka** for discovery.
Lab 5	API Gateway & Security	Implemented **Spring Cloud Gateway**; centralized **JWT Authentication** and authorization.
Lab 6	Dockerization	Containerized all services and infrastructure (DBs, Redis, RabbitMQ) into a **Docker Compose** environment.
Lab 7	Event-Driven (EDA)	Integrated **RabbitMQ**; established asynchronous communication between Payment and Notification.
Lab 8	Resilience & Performance	Configured **Resilience4j** circuit breakers and optimized routing throughput at the Gateway.
W9 & 10	Frontend & Integration	Finalized the **Flutter** mobile client; completed E2E integration testing and documentation.

Appendix B: Task Assignment & Progress Table
Note: Please update the "Assignee" column with specific group member names.

Module / Feature	**Task Description**	**Assignee**	**Status**
Backend Core	Movie, Identity, & Customer Services implementation	[Member 1]	100%
Infrastructure	API Gateway, Eureka, Config Server, & Docker Compose	[Member 2]	100%
EDA & Payment	RabbitMQ integration & Stripe Payment flow	[Member 3]	100%
Mobile Client	Flutter UI/UX, State Management, & API integration	[Member 4]	100%
Documentation	Report writing, Diagrams, & Architectural analysis	All Members	100%

Appendix C: Sample Code / Configuration

C.1 API Gateway Route Configuration (application.yml)

YAML
```
spring:
  cloud:
    gateway:
      routes:
        - id: movie-service
          uri: lb://movie-service
          predicates:
            - Path=/api/movies/**
          filters:
            - name: CircuitBreaker
              args:
                name: movieService
                fallbackUri: forward:/fallback/movie
```

C.2 Event Publishing Logic (PaymentService.java)

Java
```
public void processPayment(PaymentRequest request) {
    // Process payment with Stripe ...

    // Publish success event to RabbitMQ
    rabbitTemplate.convertAndSend(
        "payment.exchange",
        "subscription.activated",
        new SubscriptionEvent(request.getUserId(), "PREMIUM")
    );
}
```

Appendix D: System Run Guide & Repo Links

D.1 Running the System Locally

To launch the entire Nozie ecosystem, follow these steps:

1. **Prerequisites:** Ensure **Docker Desktop** and **Java 17+** are installed.
2. **Environment Setup:** Navigate to the root directory of the server.
3. **Command Execution:**

   Bash

   ```
   cd Src/server/microservices
   docker-compose up -d --build
   ```
4. **Verification:**
   - **Eureka Dashboard:** http://localhost:8761
   - **API Gateway:** http://localhost:8080
5. **Run Client:** Open the Src/client folder in VS Code/Android Studio and run flutter run.

D.2 Repository Links

- **Main Project Repo:** [Insert GitHub/GitLab Link Here]
- **Documentation Folder:** /Documents/
- **API Collection:** /Documents/Postman/Nozie_API_Collection.json